Nombre: Pedro Omar Cardona Alvarez **Nombre**: Kloudenn Jafed Jiménez Valdivieso **Nombre:** Leonardo Acosta Morales	**Matrícula**: 2226579 **Matrícula**: 2226639 **Matrícula**: 2226717
Nombre del curso: Componentes de lógica de Negocio.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad:	**Actividad**: Unidad 5 Equipo 14
Fecha: 26/10/2026	

Bibliografia (Formato APA)

- Fowler, M. (2003). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Gómez Marroquín, M. Á. (2026). Componentes de Lógica de Negocio. Microsoft Teams. UANL FIME.
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Millett, S., & Tune, N. (2015). Patterns, Principles, and Practices of Domain-Driven Design. Wrox.

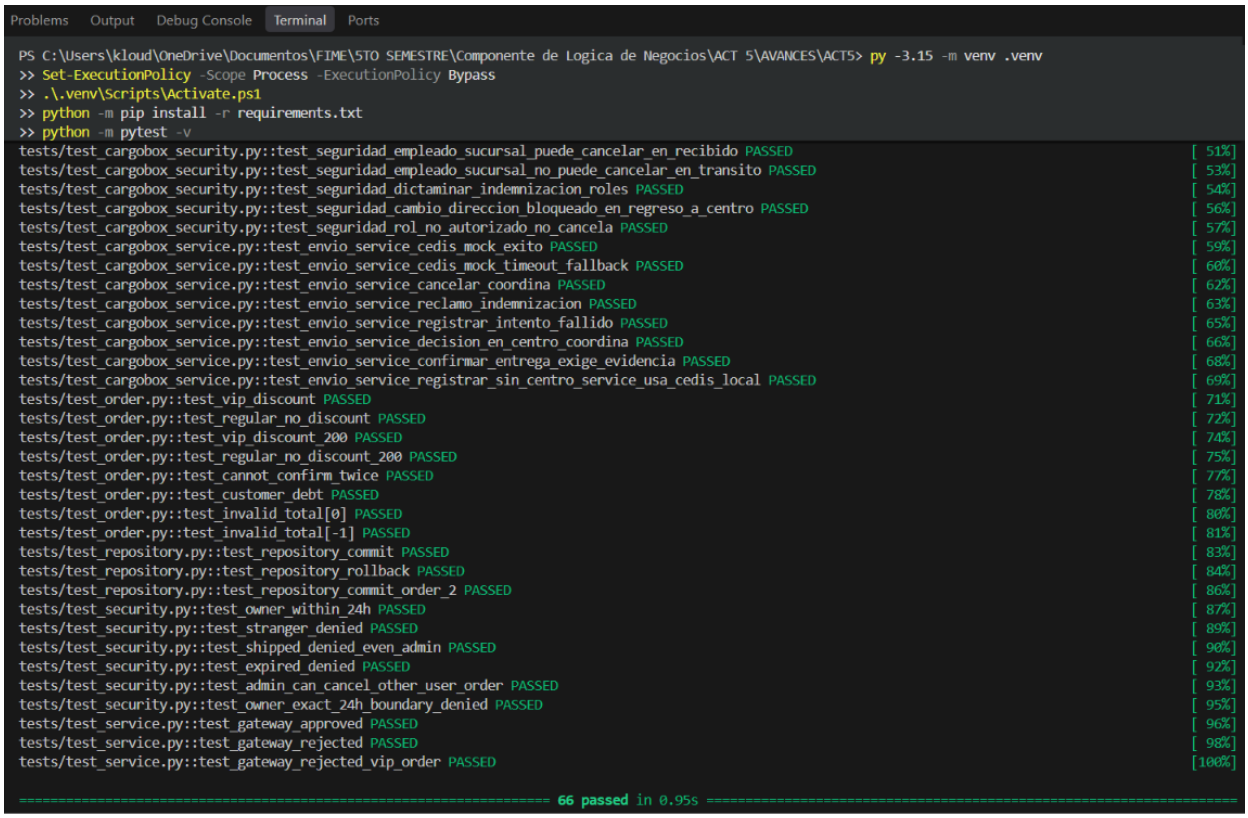
Objetivo:
Construir y ejecutar una suite de pruebas automatizadas con Python y pytest que compruebe la calidad de la lógica de negocio. Se aplican pruebas unitarias, el ciclo TDD (Red–Green–Refactor), pruebas de persistencia con SQLAlchemy y SQLite (commit y rollback), pruebas con mocks de servicios externos y pruebas de autorización. Además, se mide una cobertura mínima del 80%, se revisa la calidad del código con Ruff y se automatiza todo con un pipeline de GitHub Actions. Por último, se aplica lo aprendido a cinco reglas de negocio de la empresa CargoBox (PIA, Equipo 14).

1.- Repositorio GitHub

Enlace al repositorio: https://github.com/kloudenn29/Unidad_5

Paso 1 · Instalación y Comprobación del Proyecto Base

Comando: python -m pytest -v



Salida completa con 66 passed: pytest ejecuta automáticamente las funciones test_ y reporta si pasan. El entorno virtual aísla las librerías del proyecto

Paso 2 · Pruebas Unitarias y Estructura Arrange–Act–Assert

Comando: python -m pytest tests/test_order.py -v

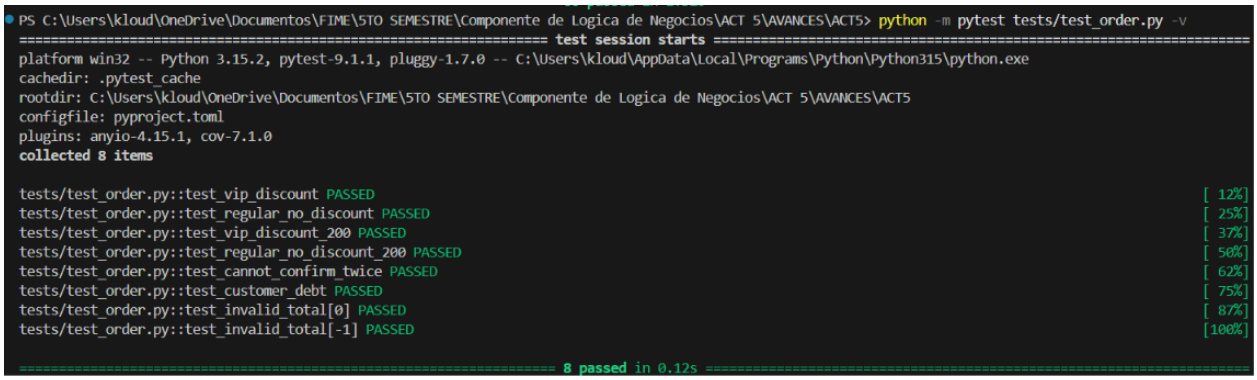


Tabla: VIP 200 → 170 y regular 200 → 200, esperado igual a real

Paso 3: Ciclo TDD Real: Red → Green → Refactor

En src/domain.py poner # al if self.status == "CONFIRMED": y a su raise.

Comando: python -m pytest tests/test_order.py::test_cannot_confirm_twice -v

```
PS C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5> python -m pytest tests/test_order.py::test_canno
t_confirm_twice -v
============================= test session starts =============================
platform win32 -- Python 3.15.2, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\kloud\AppData\Local\Programs\Python\Python315\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collected 1 item

tests/test_order.py::test_cannot_confirm_twice FAILED                    [100%]

================================== FAILURES ===================================
_____________________________ test_cannot_confirm_twice _____________________________

    def test_cannot_confirm_twice():
        """Paso 3 (TDD): Un pedido ya confirmado no puede confirmarse una segunda vez."""
        order = Order(100)
        order.confirm()
>       with pytest.raises(ValueError, match="already confirmed"):
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       Failed: DID NOT RAISE ValueError

tests\test_order.py:32: Failed
=========================== short test summary info ===========================
FAILED tests/test_order.py::test_cannot_confirm_twice - Failed: DID NOT RAISE ValueError
============================== 1 failed in 0.34s ==============================
```

Quitar los # y repitir el comando. **GREEN** (PASSED)

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5> python -m pytest tests/test_order.py::test_canno
t_confirm_twice -v
t_confirm_twice -vOneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5>
============================= test session starts =============================
platform win32 -- Python 3.15.2, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\kloud\AppData\Local\Programs\Python\Python315\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collected 1 item

tests/test_order.py::test_cannot_confirm_twice PASSED                    [100%]

============================== 1 passed in 0.11s ==============================
```

TDD: primero se escribe la prueba que falla y después el código mínimo; no es correr pruebas al final

Paso 4 · Integración con Base de Datos y Límites Transaccionales

Commando: python -m pytest tests/test_repository.py -v

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5> python -m pytest tests/test_repository.py -v
============================= test session starts =============================
platform win32 -- Python 3.15.2, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\kloud\AppData\Local\Programs\Python\Python315\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collected 3 items

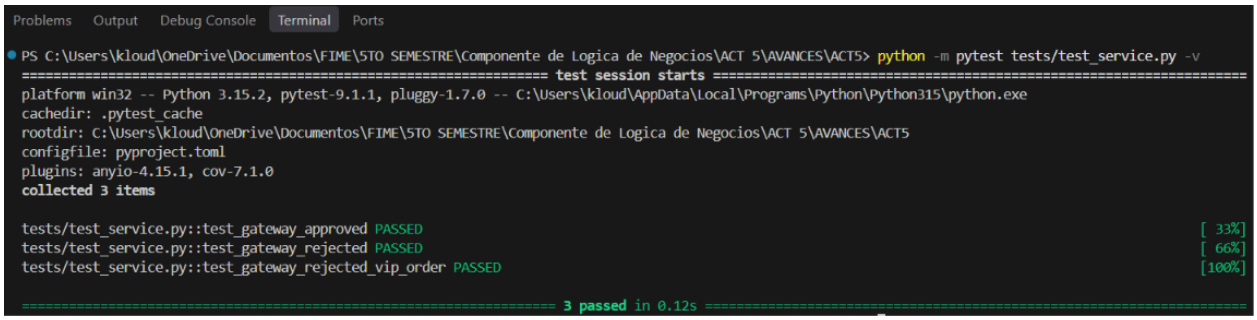
tests/test_repository.py::test_repository_commit PASSED                  [ 33%]
tests/test_repository.py::test_repository_rollback PASSED                [ 66%]
tests/test_repository.py::test_repository_commit_order_2 PASSED          [100%]

============================== 3 passed in 0.93s ==============================
```

Commit guarda el cambio de forma permanente; rollback lo descarta

Paso 5 · Servicios Externos con Dobles de Prueba (Mocks)

Commando: python -m pytest tests/test_service.py -v



El mock simula la pasarela, no hay cobro real. Riesgo: el pedido se confirma antes de cobrar. Mejora: cobrar primero y confirmar solo si el pago se aprueba.

Paso 6 · Seguridad y Autorización por Políticas

Commando: python -m pytest tests/test_security.py -v

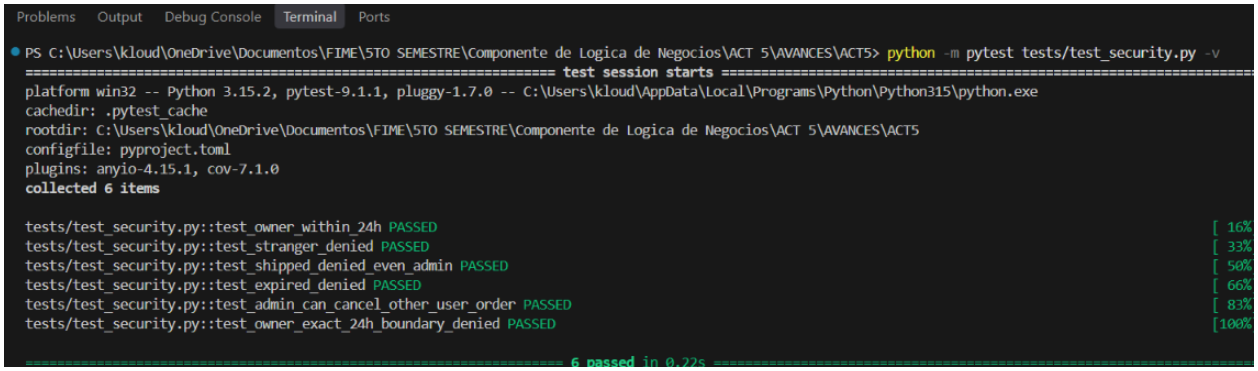


Tabla corta: admin cancela NEW ajeno → permitido, dueño a 24 h exactas → denegado, SHIPPED → denegado.

Paso 7 · Cobertura y Calidad de Código

Commando: python -m ruff check src tests

Commando: python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80

Commando: start htmlcov\index.html

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5> python -m pytest tests/test_security.py -v
>> python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80==========================================================
>> start htmlcov\index.htmlDocumentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5>
All checks passed!
============================================================ test session starts ============================================================
platform win32 -- Python 3.15.2, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\kloud\AppData\Local\Programs\Python\Python315\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kloud\OneDrive\Documentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, cov-7.1.0
collected 66 items

tests/test_cargobox_domain.py::test_cb01_cancelar_envio_en_sucursal_exito PASSED                                                    [  1%]
tests/test_cargobox_domain.py::test_cb02_cancelar_envio_en_transito_error PASSED                                                    [  3%]
tests/test_cargobox_domain.py::test_cb03_cancelar_sin_motivo_limite PASSED                                                          [  4%]
tests/test_cargobox_domain.py::test_cb04_primer_intento_fallido_reprograma PASSED                                                   [  6%]
tests/test_cargobox_domain.py::test_cb05_segundo_intento_fallido_devolucion_limite PASSED                                           [  7%]
tests/test_cargobox_domain.py::test_cb06_tercer_intento_fallido_bloqueado_error PASSED                                              [  9%]
tests/test_cargobox_domain.py::test_cb07_actualizar_direccion_en_transito_exito PASSED                                              [ 10%]
tests/test_cargobox_domain.py::test_cb08_actualizar_direccion_arribado_cedis_error PASSED                                           [ 12%]
tests/test_cargobox_domain.py::test_cb09_direccion_vacia_limite PASSED                                                              [ 13%]
tests/test_cargobox_domain.py::test_cb10_indemnizacion_con_seguro_exito PASSED                                                      [ 15%]
tests/test_cargobox_domain.py::test_cb11_indemnizacion_sin_seguro_limite PASSED                                                     [ 16%]
tests/test_cargobox_domain.py::test_cb12_peso_excedido_error PASSED                                                                 [ 18%]
tests/test_cargobox_domain.py::test_cb13_seguro_invalido_monto_negativo PASSED                                                      [ 19%]
tests/test_cargobox_domain.py::test_cb14_seguro_invalido_monto_sin_flag PASSED                                                      [ 21%]
tests/test_cargobox_domain.py::test_cb15_paquete_peso_cero_o_descripcion_vacia PASSED                                               [ 22%]
tests/test_cargobox_domain.py::test_cb16_envio_datos_obligatorios_y_flete_positivo PASSED                                           [ 24%]
tests/test_cargobox_domain.py::test_cb17_centro_cobra_reintento_extra_reprograma PASSED                                             [ 25%]
tests/test_cargobox_domain.py::test_cb18_centro_devuelve_a_sucursal_origen PASSED                                                   [ 27%]
tests/test_cargobox_domain.py::test_cb19_decision_en_centro_fuera_de_estado_error PASSED                                            [ 28%]
tests/test_cargobox_domain.py::test_cb20_entrega_con_foto_exito PASSED                                                              [ 30%]
tests/test_cargobox_domain.py::test_cb21_entrega_con_firma_digital_exito PASSED                                                     [ 31%]
tests/test_cargobox_domain.py::test_cb22_entrega_sin_evidencia_error PASSED                                                         [ 33%]
tests/test_cargobox_domain.py::test_cb23_evidencia_con_referencia_vacia_limite PASSED                                               [ 34%]
tests/test_cargobox_domain.py::test_cb24_entrega_fuera_de_reparto_error PASSED                                                      [ 36%]
tests/test_cargobox_domain.py::test_cb25_reclamo_sin_fotografia_error[None] PASSED                                                  [ 37%]
tests/test_cargobox_domain.py::test_cb25_reclamo_sin_fotografia_error[   ] PASSED                                                   [ 39%]
tests/test_cargobox_repository.py::test_cargobox_repository_commit PASSED                                                           [ 40%]
```

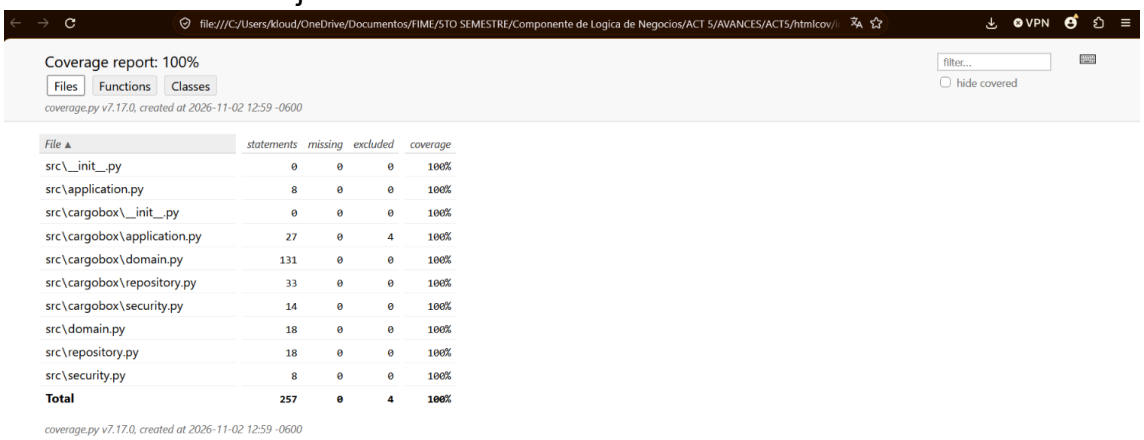
```
Problems  Output  Debug Console  Terminal  Ports

>> start htmlcov\index.htmlDocumentos\FIME\5TO SEMESTRE\Componente de Logica de Negocios\ACT 5\AVANCES\ACT5>

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================================ tests coverage ================================================================
_____________________________________________ coverage: platform win32, python 3.15.2-final-0 _____________________________________________

Name                          Stmts   Miss  Cover   Missing
-----------------------------------------------------------
src\__init__.py                   0      0   100%
src\application.py                8      0   100%
src\cargobox\__init__.py          0      0   100%
src\cargobox\application.py      27      0   100%
src\cargobox\domain.py          131      0   100%
src\cargobox\repository.py       33      0   100%
src\cargobox\security.py         14      0   100%
src\domain.py                    18      0   100%
src\repository.py                18      0   100%
src\security.py                   8      0   100%
-----------------------------------------------------------
TOTAL                           257      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
============================================================ 66 passed, 2 warnings in 1.13s ============================================================
```

Cobertura 100% de líneas. Eso no garantiza que no haya defectos, solo que todas las líneas se ejecutaron



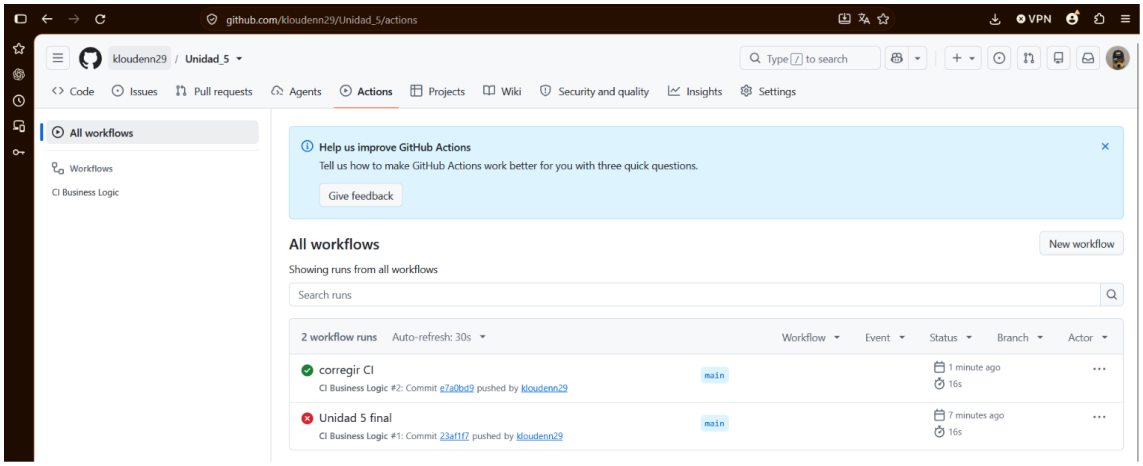
File ▲	statements	missing	excluded	coverage
src__init__.py	0	0	0	100%
src\application.py	8	0	0	100%
src\cargobox__init__.py	0	0	0	100%
src\cargobox\application.py	27	0	4	100%
src\cargobox\domain.py	131	0	0	100%
src\cargobox\repository.py	33	0	0	100%
src\cargobox\security.py	14	0	0	100%
src\domain.py	18	0	0	100%
src\repository.py	18	0	0	100%
src\security.py	8	0	0	100%
Total	**257**	**0**	**4**	**100%**

coverage.py v7.17.0, created at 2026-11-02 12:59 -0600

Paso 8 · Automatización del Flujo CI/CD con GitHub Actions

Captura de GitHub Actions: Ejecución exitosa (Verde).

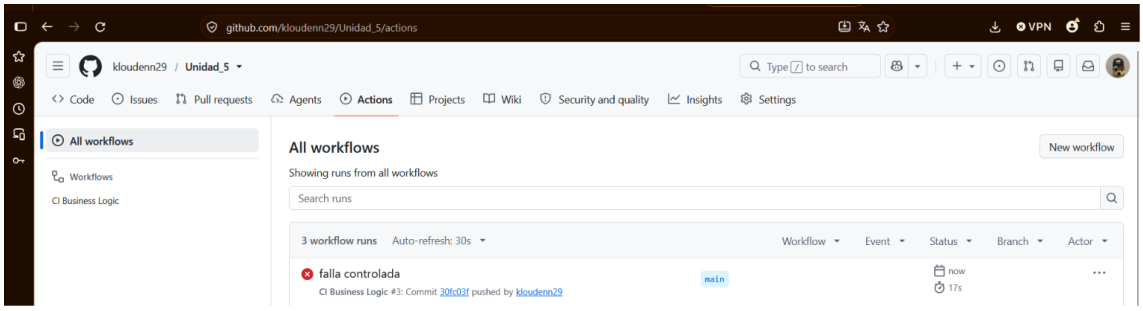
Versión inicial pasando todas las verificaciones.



Captura de GitHub Actions: Fallo controlado bloqueando el merge (Roja).

Se introdujo una aserción falsa (assert o.confirm() == 999), provocando que GitHub Actions detuviera el flujo con marca roja.

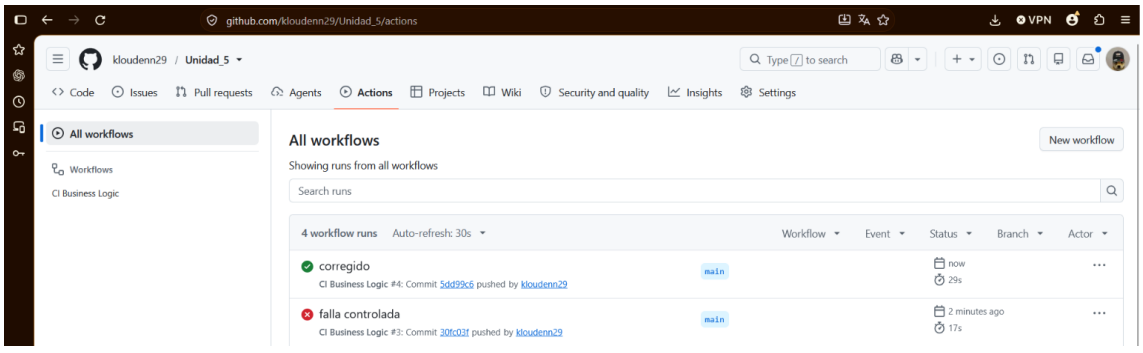
```
git commit -am "falla controlada"
git push
```



Captura de GitHub Actions: Recuperación tras corrección (Verde).

Se corrigió la aserción y el pipeline retornó a estado aprobado.

```
git commit -am "corregido"
git push
```



Paso 9 · Aplicación a la Empresa del PIA: CargoBox (Equipo 14)

Captura 1: pruebas de CargoBox

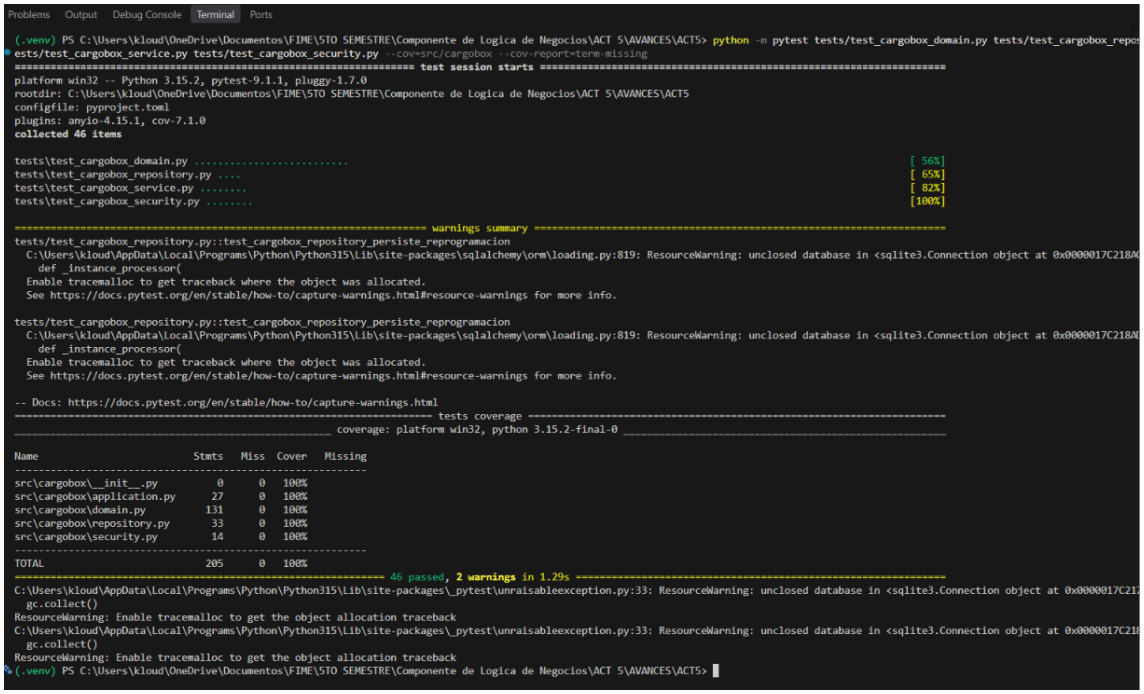
Comando: python -m pytest tests/test_cargobox_domain.py tests/test_cargobox_repository.py tests/test_cargobox_service.py tests/test_cargobox_security.py -v



Se ejecutaron 46 pruebas de las 5 reglas de CargoBox (US10, US2, US7, US11/12 y US13), todas en PASSED.”

Captura 2: cobertura

Comando: python -m pytest tests/test_cargobox_domain.py tests/test_cargobox_repository.py tests/test_cargobox_service.py tests/test_cargobox_security.py --cov=src/cargobox --cov-report=term-missing



La tabla de cobertura que ya tienes (100%). El código de CargoBox tiene 100% de cobertura de líneas, arriba del mínimo de 80%.

Matriz de casos cargobox

caso_id	entrada	resultado_esperado	tipo	estado_prueba
CP-CB01	Estado=RECIBIDO, motivo='Cliente cancelo en mostrador'	Estado cambia a CANCELADO	Exito	PASSED
CP-CB02	Estado=EN_TRANSITO, motivo='Deseo cancelar'	EnvioNoCancelableException	Error	PASSED
CP-CB03	Estado=RECIBIDO, motivo=' '	DatosEnvioInvalidosException	Limite	PASSED
CP-CB04	Estado=EN_REPARTO, motivo=DESTINATARIO_AUSENTE, intentos=0	intentos=1, estado=EN_DISTRIBUCION_DESTINO, fecha_reprogramacion= dia siguiente	Exito	PASSED
CP-CB05	Estado=EN_REPARTO, intentos_previos=1, nuevo intento	intentos=2, estado=REGRESO_CENTRO (decide cobrar reintento o devolver)	Limite	PASSED
CP-CB06	intentos_previos=2, intentar registrar tercer intento	MaximosIntentosExcedidosException	Error	PASSED
CP-CB07	Estado=EN_TRANSITO, nueva_direccion='Av. Constitucion 456'	Direccion actualizada exitosamente	Exito	PASSED
CP-CB08	Estado=EN_DISTRIBUCION_DESTINO, nueva direccion	DireccionNoActualizableException	Error	PASSED
CP-CB09	calle=' ', ciudad='Monterrey', cp='64000'	DatosEnvioInvalidosException	Limite	PASSED
CP-CB10	tiene_seguro=True, monto_declarado=12500.00, flete=350.00, foto_dano=dano_001.jpg	Indemnizacion cubre $12500.00 MXN	Exito	PASSED
CP-CB11	tiene_seguro=False, monto_declarado=0.00, flete=350.00, foto_dano=dano_002.jpg	Indemnizacion limitada a reembolso flete $350.00 MXN	Limite	PASSED
CP-CB12	peso_kg=70.1 (limite maximo operativo 70.0 kg)	DatosEnvioInvalidosException	Error/ Limite	PASSED

CP-CB13	Estado=REGRESO_CENTRO, intentos=2, cobrar_reintento=True	intentos=0, reintento_cobrado=True, estado=EN_DISTRIBUCION_DESTINO, fecha=dia siguiente	Exito	PASSED
CP-CB14	Estado=REGRESO_CENTRO, intentos=2, cobrar_reintento=False	Estado=DEVOLUCION_ORIGEN, reintento_cobrado=False	Exito	PASSED
CP-CB15	Estado=EN_REPARTO, resolver_en_centro	EnvioNoEnCentroException	Error	PASSED
CP-CB16	Estado=EN_REPARTO, evidencia=FOTO entrega_001.jpg	Estado=ENTREGADO, evidencia guardada	Exito	PASSED
CP-CB17	Estado=EN_REPARTO, evidencia=FIRMA_DIGITAL firma_001.png	Estado=ENTREGADO	Exito	PASSED
CP-CB18	Estado=EN_REPARTO, evidencia=None	SinEvidenciaEntregaException, estado sigue EN_REPARTO	Error	PASSED
CP-CB19	tipo=FOTO, referencia=''	DatosEnvioInvalidosException	Limite	PASSED
CP-CB20	Estado=EN_TRANSITO, evidencia=FOTO	EntregaNoPermitidaException	Error	PASSED

Se probaron 5 reglas de CargoBox: US10 (máximo 2 intentos, luego el paquete regresa al centro de distribución, donde se decide cobrar reintento o devolver a origen), US2 (cancelar solo en sucursal origen), US7 (cambiar dirección solo antes de llegar al centro destino), US11/12 (con seguro se paga el valor declarado, sin seguro solo el flete, y la foto del daño es obligatoria) y US13 (entregar solo con foto o firma digital). Cada regla tiene casos de éxito, error y límite. Las reglas viven en el dominio, la persistencia se probó con SQLite en memoria (commit, rollback y unicidad) y no se usaron servicios externos reales

Vibe Coding con IA: Registro y Uso Responsable

Elemento	Registro
Herramienta	CLAUDE IA, usada para proponer casos de prueba y la estructura en pytest
Prompt	"Analiza la entidad Envio de CargoBox y las reglas de cancelación y reintentos. Propón casos de prueba pytest para éxito, error y valores límite. No inventes reglas que contradigan el BRD de la Unidad 2."
Resultado de la IA	Casos de cancelación, límite de intentos y roles. Sugirió que un Supervisor cancelara un envío en tránsito
Caso rechazado	Se rechazó porque contradice el BRD y la US2: al salir de la sucursal, la cancelación es inviable por cadena de custodia
Corrección manual	Se mantuvo la regla del BRD: se lanza `EnvioNoCancelableException`
Prueba añadida	`test_cb02_cancelar_envio_en_transito_error` y `test_seguridad_empleado_sucursal_no_puede_cancelar_en_transito`

Reflexión Final del Aprendizaje

Durante el desarrollo de esta preactica se pudo comprender la importancia de hacer pruebas automatizadas para garantizar la calidaad y el correcto funcionamiento de la lógica de negocio del sistema. Al igual que, aprendimos a usar phyton y phytest para diseñar y ejecutar diferentes tipos de pruebas, asi como identificar errores antes de que afecten el funcionamiento de la aplicación. Tambien se aprendio el ciclo TDD (Red-Green-Refractor) y su utilidad, ya que permite desarollar codigo de manera mas organizada, comprobando primero los errores y luego realizando las correcciones necesarias.

Ademas que, aprendimos como funcionan las pruebas de persistencia con SQLAlchemy y SQLit, el uso de mocks para simular servicios externos y la implementación de pruebas de seguridad para verificar que los usuarios tengan los permisos correspondientes. Otro aprendizaje que adquirimos fue conocer la utilidad de herramientas como Ruf, la mediación de cobertura de código y GitHub Actions para automatizar las verificaciones y mantener un estandár de calidad durante el desarrollo.

Finalmente, al aplicar los conocimientos que adquirimos al proyecto de CargoBox, pudimos comprender mejor cómo transladar las reglas de negocio a pruebas concretas que contemplen escenarios exitosos, errores y valores límite.

Tambien aprendimos que el uso de la Inteligencia Artificial (IA) nos puede facilitar el desarrollo y proponer ideas, pero aun es necesarios revisar sus resultados ty comprobar que respeten los requerimientos dados.